Instituto Federal de Minas Gerais

Campus São João Evangelista

Bacharelado em Sistemas de Informação

Linguagem de Programação I

Aula 8 Exercícios de Aprendizagem

Luiz Andre Fernandes Pego

Turma SI251

São João Evangelista

Outubro de 2026

Exercício 1

Na aula anterior, você implementou uma classe Stack em Java, com operações básicas como empilhar, desempilhar, verificar se está vazia e tratamento de exceções para situações como tentativa de desempilhar de uma pilha vazia. Nesta atividade, adicione um construtor de cópia à classe Stack, de forma que seja possível criar uma nova pilha a partir de outra já existente, copiando todos os elementos na mesma ordem. Após implementar o construtor, crie uma classe de teste que demonstre seu funcionamento. No método main, crie uma pilha original, adicione alguns elementos e, em seguida, utilize o construtor de cópia para criar uma nova pilha baseada na primeira. Mostre, por meio de saídas no console, que ambas as pilhas contêm os mesmos valores, mas são objetos independentes (ou seja, alterações em uma não afetam a outra).

```
50 40 30 20 10
50 40 30 20 10
```

Exercício 1: saída do programa original no VS Code.

Exercício 2

Amplie a classe Stack criada anteriormente, implementando um construtor que receba um número inteiro n como parâmetro. Esse construtor deve inicializar a pilha vazia e, em seguida, empilhar automaticamente os números inteiros de 1 até n, nessa ordem (ou seja, o número 1 é empilhado primeiro e o número n ficará no topo). Após implementar o construtor, crie um programa de teste no método main que solicite ao usuário um valor n, crie uma nova pilha utilizando o construtor implementado e exiba os elementos empilhados. Em seguida, desempilhe todos os elementos mostrando a sequência de remoção, para confirmar que a pilha está funcionando corretamente e que os elementos foram empilhados na ordem esperada.

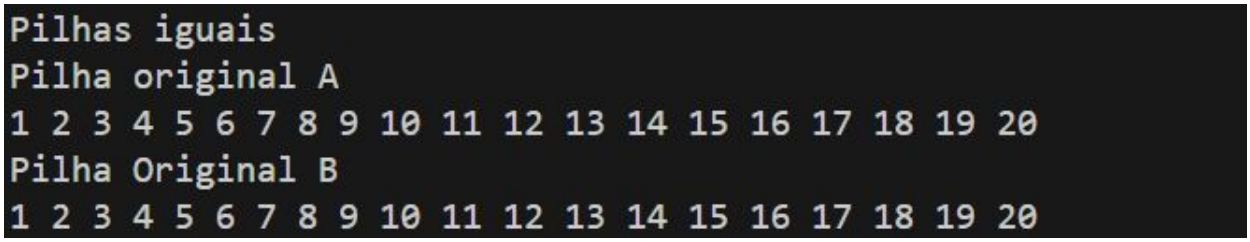
```
Digite um valor

5
Valores originais exercicio 2:
5 4 3 2 1
Valores removidos um a um
4 3 2 1
3 2 1
2 1
1
```

Exercício 2: entrada 5 e saída do programa original no VS Code.

Exercício 3

Agora, modifique a a classe Stack já desenvolvida, incluindo um método estático chamado equals que receba como parâmetros dois objetos do tipo Stack e retorne um valor lógico (true ou false) indicando se as pilhas são iguais ou diferentes. Duas pilhas devem ser consideradas iguais se possuírem o mesmo número de elementos e se todos os elementos corresponderem exatamente na mesma ordem, do topo até a base. O método não deve alterar o conteúdo das pilhas comparadas; caso seja necessário percorrer os elementos, utilize pilhas auxiliares para preservar o estado original. No método main, crie duas instâncias da classe Stack, empilhando alguns valores em cada uma delas (você pode utilizar o construtor que empilha automaticamente os números de 1 até n). Em seguida, invoque o método equals para comparar as duas pilhas e exiba uma mensagem informando se elas são iguais ou diferentes.



Exercício 3: resultado da comparação e conteúdo exibido pelo programa original.

Exercícios 1 a 3

```
public class Node {

    private int item;
    private Node next;
    public void setItem(int item) {
    this.item = item;
    }
    public void setNext(Node next) {
    this.next = next;
    }
    public int getItem() {
    return this.item;
    }
    public Node getNext() {
    return this.next;

    }
    }
```

Exercícios 1 a 3

```
import java.util.EmptyStackException;

public class Stack {
    private Node top;
    private int n;

    public Stack() {
        this.top = null;
        this.n = 0;
    }
```

```
// Construtor exercicio 2
public Stack(int n) {
    for (int i = 1; i < n + 1; i++) {
        this.push(i); // Empilha os valores de 1 ate n
    }
}

// Construtor copia-exercicio 1
public Stack(Stack outra, int quantidade) {
    Node nn = outra.top;
    int i = 0;
    int[] valores = new int[quantidade];

    while (nn != null) {
        valores[i] = nn.getItem();

        if (nn.getNext() == null) {
            break;
        }
        i++;
        nn = nn.getNext();
    }
    for (i = quantidade; i != 0; i--) {
        this.push(valores[i - 1]);
    }
}

// Exercicio 3
public static boolean equal(Stack a, Stack b, Stack Original_A, Stack Original_B) {
    int aux = 0;
    int aux2 = a.size();
    if (a.size() != b.size()) {
        return false;
    } else // A duas pilhas tem o mesmo tamanho
    {
        for (Node t = a.top; t != null; t = t.getNext()) {
            if (a.getTop() == b.getTop()) {
                aux++;
            }
            Original_A.push(a.getTop());
            Original_B.push(b.getTop());
            a.pop();
            b.pop();
        }

        if (aux == aux2) {
            return true;
        }

        return false;
    }
}

public void push(int item) {
    Node novo = new Node();
    novo.setItem(item);
    novo.setNext(this.top);
    this.top = novo;
    this.n++;
}
```

```
    public int pop() {
        if (this.isEmpty()) {
            throw new EmptyStackException();
        }
        int valor = this.top.getItem();
        this.top = this.top.getNext();
        this.n--;
        return valor;
    }

    public int getTop() {
        if (this.isEmpty()) {
            throw new EmptyStackException();
        }
        return this.top.getItem();
    }

    public boolean isEmpty() {
        return this.n == 0;
    }

    public int size() {
        return this.n;
    }

    public void clear() {
        this.top = null;
        this.n = 0;
    }

    public void show() {
        Node novo = this.top;
        while (novo != null) {
            System.out.print(novo.getItem() + " ");
            novo = novo.getNext();
        }
        System.out.println();
    }
}
```

Exercícios 1 a 3

```
import java.util.EmptyStackException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Stack pilha = new Stack();
        pilha.push(10);
        pilha.push(20);
        pilha.push(30);
        pilha.push(40);
        pilha.push(50);

        // Exercicio 1

        Stack copia = new Stack(pilha, pilha.size());
        // Objeto original
        pilha.show();
        // Remove um elemento da pilha e prova que são independentes pois não vai
        // auterar a do objeto copia=
        pilha.pop();
        // Objeto copiado
        copia.show();

        // Exercicio 2
        Scanner input = new Scanner(System.in);

        int valor = 0;
        System.out.println("Digite um valor\n");
        valor = input.nextInt();

        Stack exercicio_2 = new Stack(valor);
        System.out.println("Valores originais exercicio 2:");
        exercicio_2.show();

        System.out.println("Valores removidos um a um");
        for (int i = 0; i < valor; i++) {
            exercicio_2.pop();
            exercicio_2.show();
        }

        //Exercicío 3

        Stack aux1 = new Stack(20);
        Stack aux2 = new Stack(20);

        Stack Original_A = new Stack();
        Stack Original_B = new Stack();

        Stack main = new Stack();

        if(main.equal(aux1,aux2, Original_A, Original_B))
        {
            System.out.println("Pilhas iguais");
        }
        else
        {
            System.out.println("Pilhas diferentes");
        }
        System.out.println("Pilha original A");
        Original_A.show();
```

```
        System.out.println("Pilha Original B");
        Original_B.show();
    }
}
```

Exercício 4

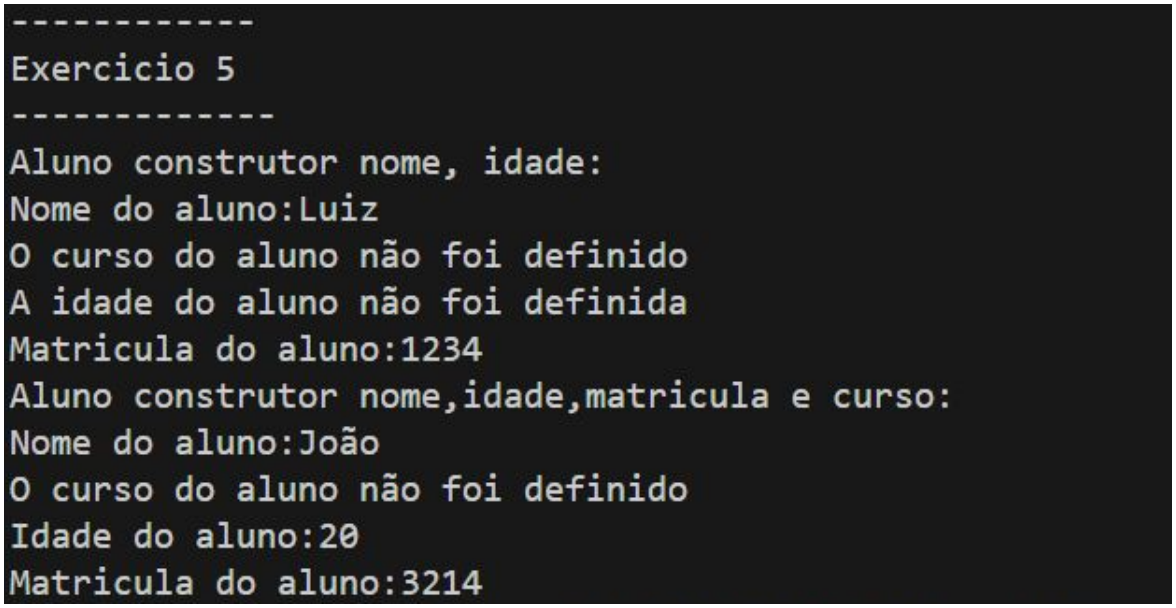
Crie um programa em Java com classes de um sistema simples de gerenciamento de alunos de uma universidade. Cada aluno deve ser representado por uma classe contendo atributos como nome, matricula, curso e idade. Utilize o conceito de encapsulamento para garantir que os atributos só possam ser acessados por meio de métodos públicos (getters e setters). Em seguida, crie um método para exibir as informações do aluno de forma formatada. No método main, instancie pelo menos três objetos da classe Aluno e exiba seus dados na tela.

```
Aluno 1
Nome do aluno:Luiz
Curso do aluno:Sistemas de informação
Idade do aluno:19
Matricula do aluno:1234
Aluno 2
Nome do aluno:Pedro
Curso do aluno:Ciências da computação
Idade do aluno:20
Matricula do aluno:4321
Aluno 3
Nome do aluno:João
Curso do aluno:Engenharia de software
Idade do aluno:21
Matricula do aluno:3124
```

Exercício 4: dados dos três alunos na execução original.

Exercício 5

Amplie o exercício anterior incluindo construtores sobrecarregados na classe Aluno: um construtor sem parâmetros, um que receba apenas nome e matrícula, e outro que receba todos os atributos. No programa principal, crie objetos utilizando cada um dos construtores e verifique como a sobrecarga facilita a criação de instâncias com diferentes quantidades de informações.



Exercício 5: saída dos construtores na execução original.

Exercícios 4 e 5

```
public class Alunos {

    private String nome;
    private int matricula;
    private String curso;
    private int idade;

    public Alunos()
    {
        this.nome = null;
        this.matricula = 0;
        this.curso = null;
        this.idade = 0;
    }

    //Exercicio 5
    public Alunos(String nome, int matricula)
    {
        this.setNome(nome);
        this.setMatricula(matricula);
    }

    public Alunos(String nome, int matricula, String Curso, int idade)
    {
        this.setNome(nome);
        this.setIdade(idade);
        this.setMatricula(matricula);
        this.setNome(nome);
    }

    public void setCurso(String curso) {
        this.curso = curso;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }
```

```
    public void setMatricula(int matricula) {
        this.matricula = matricula;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getCurso() {
        return curso;
    }

    public int getIdade() {
        return idade;
    }

    public int getMatricula() {
        return matricula;
    }

    public String getNome() {
        return nome;
    }
}
```

Exercícios 4 e 5

```
public class Exibir {
    public static void Exibir(Alunos a) {
        if (a.getNome() != null) {
            System.out.println("Nome do aluno:" + a.getNome());
        }
        else
        {
            System.out.println("O nome do aluno esta não foi definido");
        }
        if(a.getCurso() !=null)
        {
            System.out.println("Curso do aluno:" + a.getCurso());
        }
        else
        {
            System.out.println("O curso do aluno não foi definido");
        }
        if(a.getIdade() !=0)
        {
            System.out.println("Idade do aluno:" + a.getIdade());
        }
        else
        {
            System.out.println("A idade do aluno não foi definida");
        }
        if(a.getMatricula() !=0)
        {
            System.out.println("Matricula do aluno:" + a.getMatricula());
        }
        else
        {
            System.out.println("A matricula do aluno não foi definida");
        }
    }
}
```

Exercícios 4 e 5

```
public class Main {
   public static void main(String[] args) {
      //Exercicio 4

      Exibir aux = new Exibir();
      Alunos a1 = new Alunos(); // Objeto 1
      Alunos a2 = new Alunos(); // Objeto 2
      Alunos a3 = new Alunos(); // Objeto 3

      a1.setCurso("Sistemas de informação");
      a2.setCurso("Ciências da computação");
      a3.setCurso("Engenharia de software");

      a1.setIdade(19);
      a2.setIdade(20);
      a3.setIdade(21);

      a1.setMatricula(1234);
      a2.setMatricula(4321);
      a3.setMatricula(3124);
```

```
        a1.setNome("Luiz");
        a2.setNome("Pedro");
        a3.setNome("João");

        System.out.println("Aluno 1");
        aux.Exibir(a1);
        System.out.println("Aluno 2");
        aux.Exibir(a2);
        System.out.println("Aluno 3");
        aux.Exibir(a3);

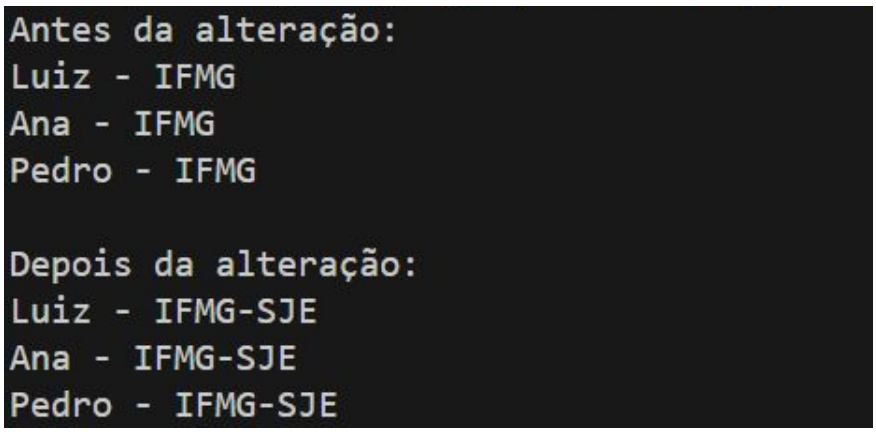
        //Exercicio 5

        Alunos a4 = new Alunos("Luiz", 1234);
        Alunos a5 = new Alunos("João", 3214,"Sistemas de informação", 20);
        System.out.println("------------");
        System.out.println("Exercicio 5");
        System.out.println("-------------");
        System.out.println("Aluno construtor nome, idade:");
        aux.Exibir(a4);
        System.out.println("Aluno construtor nome,idade,matricula e curso:");
        aux.Exibir(a5);

    }
}
```

Exercício 6

Crie uma classe chamada Universidade com um atributo estático que armazene o nome da instituição. Faça com que cada objeto da classe Aluno utilize esse atributo para exibir, junto de suas informações, o nome da universidade. Além disso, crie um método estático na classe Universidade que permita alterar o nome da instituição e verifique como essa alteração afeta todos os objetos já criados.



Exercício 6: nome da instituição antes e depois da alteração.

Exercício 6

```
public class Aluno {
    private String nome;
    private String matricula;
    private String curso;
    private int idade;
```

```
    public Aluno() {
    }

    public Aluno(String nome, String matricula) {
        this.nome = nome;
        this.matricula = matricula;
    }

    public Aluno(String nome, String matricula, String curso, int idade) {
        this.nome = nome;
        this.matricula = matricula;
        this.curso = curso;
        this.idade = idade;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getMatricula() {
        return matricula;
    }

    public void setMatricula(String matricula) {
        this.matricula = matricula;
    }

    public String getCurso() {
        return curso;
    }

    public void setCurso(String curso) {
        this.curso = curso;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }

    public void mostrar() {
        System.out.println("Nome: " + nome);
        System.out.println("Matrícula: " + matricula);
        System.out.println("Curso: " + curso);
        System.out.println("Idade: " + idade);
        System.out.println("Universidade: " + Universidade.getNome());
    }
}
```

Exercício 6

```
public class Universidade {
    private static String nome;
    private int alunos_quantidade;
    private String cidade;
    private int ano_fundaocao;

    public static void  auterar_nome(String nome)
    {
        Universidade.nome = nome;
    }

    public void setAlunos_quantidade(int alunos_quantidade) {
        this.alunos_quantidade = alunos_quantidade;
    }

    public void setAno_fundaocao(int ano_fundaocao) {
        this.ano_fundaocao = ano_fundaocao;
    }

    public void setCidade(String cidade) {
        this.cidade = cidade;
    }

    public static void setNome(String nome) {
        Universidade.nome = nome;
    }

    public int getAlunos_quantidade() {
        return alunos_quantidade;
    }

    public int getAno_fundaocao() {
        return ano_fundaocao;
    }

    public String getCidade() {
        return cidade;
    }

    public static String getNome() {
        return nome;
    }

}
```

Exercício 6

```
public class Main {
    public static void main(String[] args) {
        Universidade.setNome("IFMG");

        Aluno a = new Aluno();
        Aluno b = new Aluno();
        Aluno c = new Aluno();

        a.setNome("Luiz");
        b.setNome("Ana");
        c.setNome("Pedro");

        Aluno[] alunos = {a, b, c};

        System.out.println("Antes da alteração:");
        for (Aluno aluno : alunos) {
            System.out.println(aluno.getNome() + " - " + Universidade.getNome());
        }

        Universidade.auterar_nome("IFMG-SJE");

        System.out.println("\nDepois da alteração:");
        for (Aluno aluno : alunos) {
            System.out.println(aluno.getNome() + " - " + Universidade.getNome());
        }
    }
}
```

Exercício 7

Explique, com suas próprias palavras, os conceitos de checked exceptions e unchecked exceptions na linguagem Java, destacando as diferenças entre esses dois tipos de exceção e apresentando exemplos que ilustrem cada caso. Em seguida, discuta de que forma a distinção entre exceções verificadas e não verificadas pode contribuir para a adoção de boas práticas de programação, como a escrita de códigos mais seguros, organizados e fáceis de manter. Seu texto deve evidenciar a importância do tratamento adequado de erros para garantir maior confiabilidade e estabilidade na execução de aplicações Java.

Cheked exceptions são usadas para erros previsiveis e devem ser obrigatoriamente tratadas no método ou por quem chamar o método. Já uncheked exceptions representam erros de lógica e não é obrigatório tratá-las

Exercício 7

```
public class Exercicio_7 {

    public void divisao(int a, int b)
    {
        try{
            System.out.println(a/b);
        }catch(ArithmeticException e){
            System.out.println("Erro ao dividir por zero");
        }
    }
}
```

Exercício 7

```
public class Main {
    public static void main(String[] args) {
        Exercicio_7 aux = new Exercicio_7();

        aux.divisao(10, 0);
        aux.divisao(5, 5);
    }
}
```

Exercício 7

```
public class Main2 {
    public static void main(String[] args) {

        try {
            System.out.println("Iniciando...");

            Thread.sleep(3000);

            System.out.println("Finalizado!");

        } catch (InterruptedException e) {
            System.out.println("Ocorreu um erro!");
        }
    }
}
```

```
Erro ao dividir por zero
1
```

Exercício 7: divisão por zero e execução da divisão seguinte.

```
Iniciando...
Finalizado!
```

Exercício 7: início e término da pausa de três segundos.

Exercício 8

Explique, com suas próprias palavras, a diferença entre o uso das palavras-chave throw e throws no contexto do tratamento de exceções em Java.

Quando usamos throw lançamos a exceção necessária nesse momento. Já quando throws é usado falamos que quem chamar esse método deve tratar a exceção.